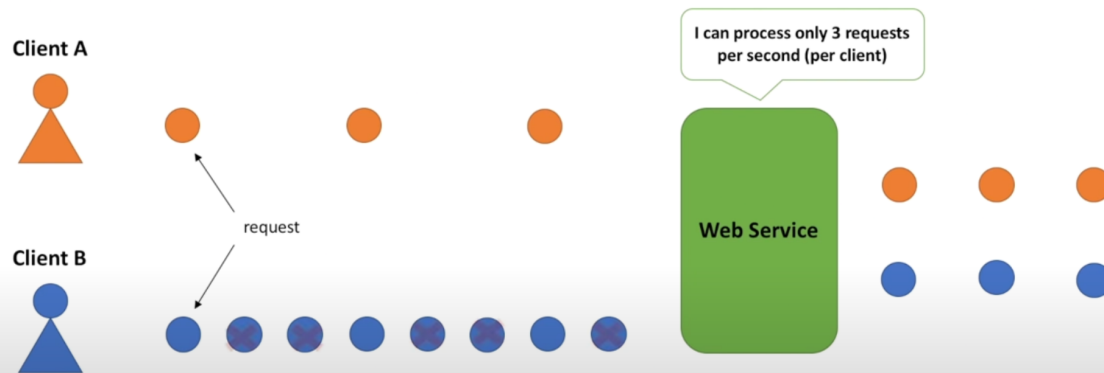


# Rate Limiting

 [System Design Interview - Rate Limiting \(local and distributed\)](#)

## Problem Statement



## Requirements Clarification

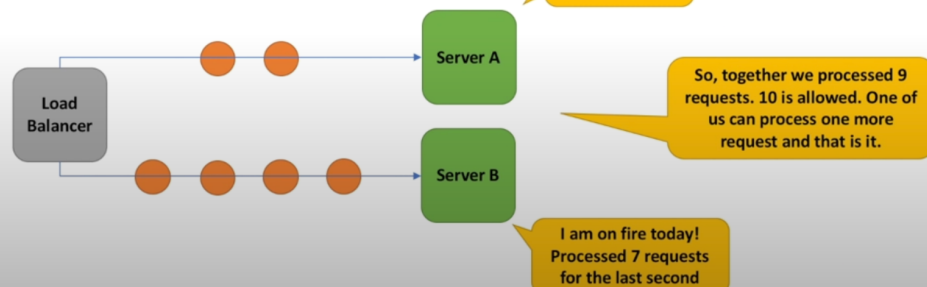
Do not we need to write a software that scales to handle high load?



What about max connections in the Load Balancer and max threads count on the service endpoint. Do we still need throttling?



Ok, I agree we might need throttling. But it does not seem like a distributed system problem. What if we throttle on each host individually?



## Requirements

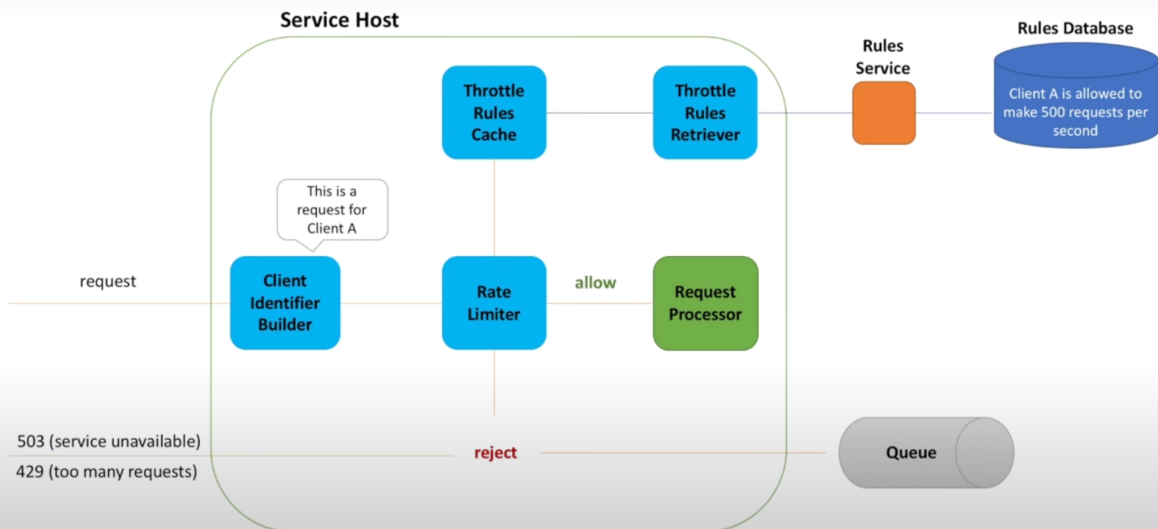
### Functional

- `allowRequest(request)`

### Non-Functional

- **Low Latency** (make decision as soon as possible)
- **Accurate** (as accurate as we can get)
- **Scalable** (supports an arbitrarily large number of hosts in the cluster)

## Request Processing



## Token Bucket Algorithm



## Token Bucket Algorithm

```
public class TokenBucket {
    private final long maxBucketSize;
    private final long refillRate;

    private double currentBucketSize;
    private long lastRefillTimestamp;

    public TokenBucket(long maxBucketSize, long refillRate) {
        this.maxBucketSize = maxBucketSize;
        this.refillRate = refillRate;

        currentBucketSize = maxBucketSize;
        lastRefillTimestamp = System.nanoTime();
    }

    public synchronized boolean allowRequest(int tokens) {
        refill();

        if (currentBucketSize > tokens) {
            currentBucketSize -= tokens;
            return true;
        }

        return false;
    }

    private void refill() {
        long now = System.nanoTime();
        double tokensToAdd = (now - lastRefillTimestamp) * refillRate / 1e9;
        currentBucketSize = Math.min(currentBucketSize + tokensToAdd, maxBucketSize);
        lastRefillTimestamp = now;
    }
}
```

Number of tokens initially is equal to the maximum capacity

Current time in nanoseconds

Synchronized, as several threads may be calling the method concurrently

First, refill bucket with tokens accumulated since the last call

If bucket has enough tokens, call is allowed

Request is throttled as bucket does not have enough tokens

These many tokens accumulated since the last refill

Number of tokens should never exceed maximum capacity

## Token Bucket Algorithm Example

10 max, 10/sec



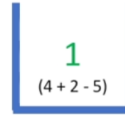
t0

allowRequest(6)



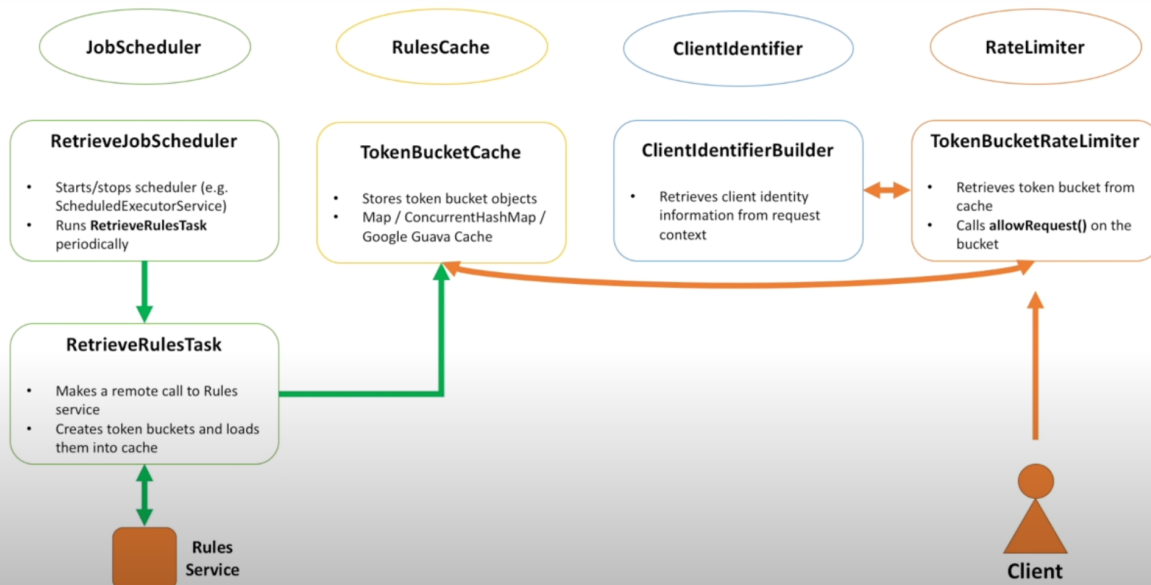
t1 = t0 + 300 ms

allowRequest(5)



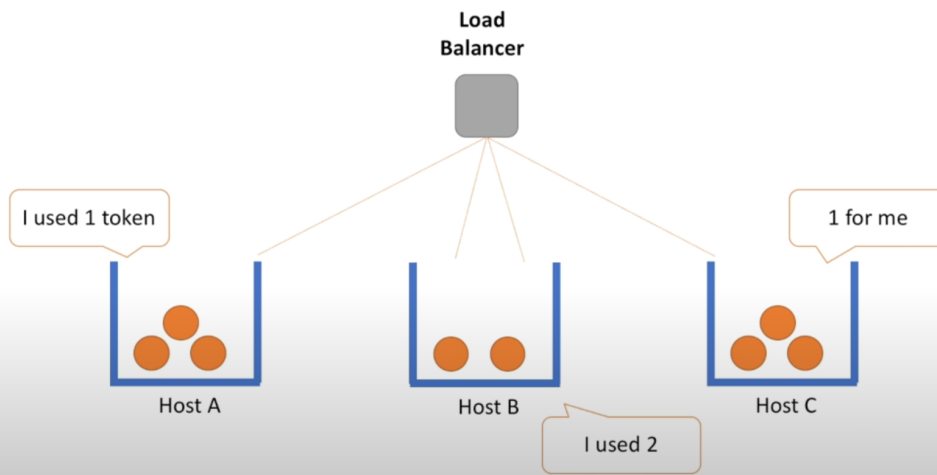
t2 = t1 + 200 ms

## Interfaces and Classes



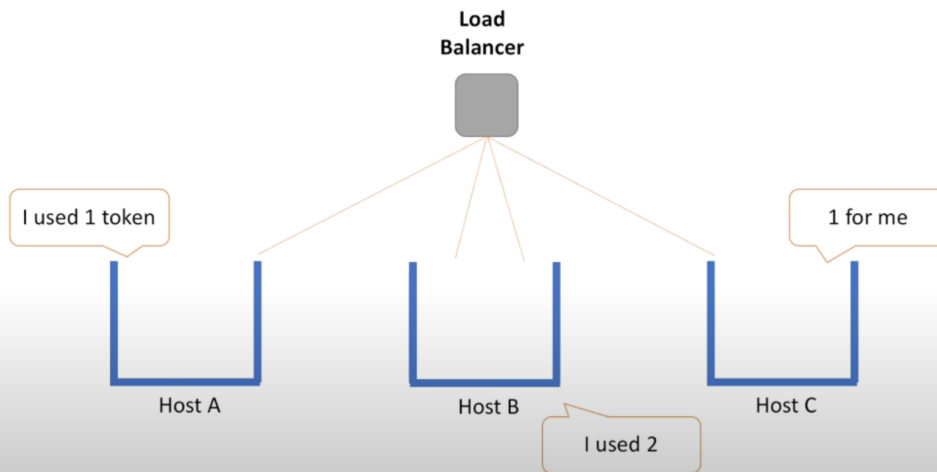
## Stepping into the distributed world

Service can handle 4 requests per second (per client)



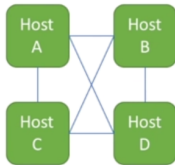
## Stepping into the distributed world

Service can handle 4 requests per second (per client)

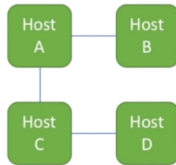


## Message Broadcasting

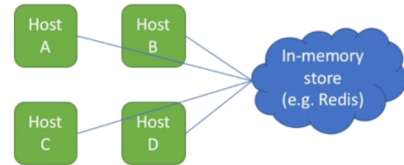
Tell everyone everything



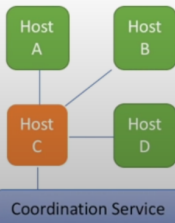
Gossip communication



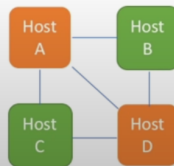
Distributed cache



Coordination service



Random leader selection



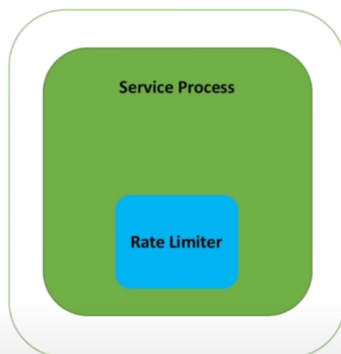
And what about communication protocols. What should we choose?



TCP vs UDP

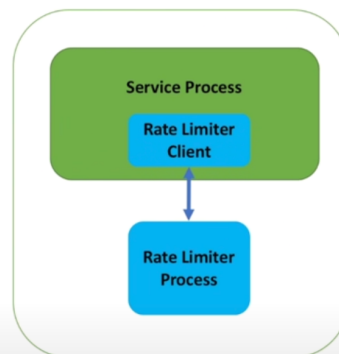
## How to integrate all this with the service?

Service Host



- Faster
- Resilient to inter-process call failures

Service Host



- Programming language agnostic
- Uses its own memory space
- Makes it easier to deal with service teams paranoia

## What else can an interviewer ask?

My service is insanely popular, millions of users. Does this mean millions of buckets are stored in memory?

Please tell me more about failure scenarios.



Interviewer

I am a little bit concerned about synchronization. Is not it a bottleneck?

Let's talk about rules configuration. This may become a nightmare to manage, right?



Interviewer

What should clients do when their requests are throttled?

## One final look

